Learning to cooperate with multiplicative updates

Phillip Nicol

July 15, 2020

Abstract

Understanding how self-interested agents can learn to interact optimally in strategic situations is an important problem in both game theory and machine learning. A common approach is to use a no-regret algorithm, which guarantees that no other action could have resulted in a higher payoff for the learner in hindsight. However, the no-regret framework fails to account for the fact that the opponent may react to the learner's actions. In the case of a reactive opponent, we show that no-regret algorithms can lead to sub-optimal play even for simple games. To address this issue, we develop the DU-Hedge algorithm. DU-Hedge is a simple variant of the Hedge algorithm that is able to best-respond against simple reactive opponents. When the opponent is unresponsive, we show that the behavior of DU-Hedge is similar to that of Hedge and prove a convergence result. In the prisoner's dilemma we show that DU-Hedge is able to cooperate even in self-play. We supplement our findings with an in-depth review of Hedge as a no-regret learning algorithm for playing a game.

Introduction

The central goal of game theory is to predict how self-interested agents will interact in strategic situations. Oftentimes, the agent begins with no knowledge of the game and therefore must learn to play optimally through repeated play. This learning task differs from others in that environment is not static: the actions of the learner influences the behavior of her opponent [8].

A common approach for learning a game is to use a regret minimization algorithm [10,11]. Informally, regret is defined as the payoff that the learner "missed" by not choosing to play a different action in hindsight. Under standard assumptions, the learner is able to observe the payoffs that other actions would have received after the fact so that the regret is explicitly known by the learner. There are several algorithms that are known to achieve an average regret of zero in the limit [13]. This property of vanishing regret is known as *no-regret*. One of the most well-known no-regret algorithms is Hedge, which uses multiplicative updates to assign higher weights to actions that receive a higher payoff. An in-depth review of Hedge is given in the preliminaries section.

The glaring flaw in the no-regret framework is that it assumes the opponent is completely unresponsive. That is, the definition of regret assumes that the learner's action do not influence the strategy of her opponent. However, this assumption is unlikely to hold even when both agents are using a no-regret algorithm to choose actions. Consider the prisoner's dilemma game in figure 1, where the learner has the option to either cooperate (C) or defect (D). It is easy to see that no matter what the opponent's action is, it is best for the other player to defect. That is, defection is a strictly dominant action in the prisoner's dilemma. However, if players could somehow agree to cooperate, they could obtain a higher payoff in the long run. In the repeated prisoner's dilemma, there are equilibria in which cooperation is always played [13]. For instance, consider the *tit-for-*

tat strategy where the player chooses C if and only her opponent chose C on the previous iteration of the game. If the learner used HEDGE against an opponent using the tit-for-tat strategy, the learner would have no regrets about always playing D (since it is always the best action in hindsight). However, if the learner assumes that the opponent reacts to her action, she would clearly have regrets about not playing C.

Another example of the failure of no-regrets to account for reactive opponents is given by [19]. Consider the coordination game given in figure 1, where both players can achieve a payoff of 10 by playing action A. This game is simpler than the prisoner's dilemma in the sense that there is no incentive for a player to deviate from A if she believes the other player also plans on playing A. Consider the learner facing an opponent who simply plays whatever the learner did in the last round. Then it is possible for the learner to have no-regrets (in the time-averaged sense) from playing B every round, although she could have obtained a much higher payoff by playing A.

The goal for this paper is to address the inability of no-regret algorithms such as HEDGE to best respond to reactive opponents. Unfortunately, [19] goes on to show that it is impossible to design a regret minimization algorithm under this notion of "adaptive" regret. The problem is that reactive opponents strategies can be so complicated that no learning algorithm could possibly account for all of them. Thus, it is necessary that we refine our goal to include "reasonably simple" opponent strategies, such as those that can be implemented by finite automata with very few states [4,16].

After a thorough review of HEDGE and its convergence properties, we introduce the Delayed Update Hedge (DU-HEDGE) algorithm to address the shortcomings explained above. DU-HEDGE maintains the favorable time and space complexity of HEDGE but has the ability to best respond against simple reactive opponent strategies, such as the tit-for-tat strategy. In the event of a completely unresponsive opponent, we provide experimental evidence that DU-HEDGE behaves similarly to HEDGE and prove convergence in probability to the best-response action for a certain subclass of games. With the addition of a simple search heuristic, we show that DU-HEDGE can occasionally cooperate in the prisoner's dilemma even when playing against a copy of itself.

Prisoner's Dilemma

		Player 2	
		C	**D**
Player 1	**C**	$3,3$	$0,5$
	D	$5,0$	$1,1$

Coordination game

		Player 2	
		A	**B**
Player 1	**A**	$10,10$	$0,0$
	B	$0,0$	$1,1$

Figure 1: Prisoner's dilemma and a coordination game. Player 1's payoff comes first.

Related work. The inability of HEDGE to cooperate in the prisoner's dilemma has been frequently noted in literature [4, 13, 19]. [7] designed an algorithm that can best respond to tit-for-tat by using an exploration-exploitation heuristic that forces the learner to at least try cooperation before immediately converging to defection. [4] models the opponent as a Markov chain where the strategy depends on the previous action of the learner. DU-HEDGE obtains cooperation without explicitly modeling the opponent, which is more memory efficient.

Neither of the above approaches address the question of obtaining cooperation in self-play. This problem is significantly more difficult since one can not rely on the opponent to have a refined strategy from the outset (since the game is unknown to *both* the learner and the opponent at the start). [3] uses an exploration-exploitation approach combined with reinforcement learning techniques to obtain self-play cooperation in certain variants of the prisoner's dilemma. The methods of [3] require both players to store the entire history of the game in memory. On the contrary, DU-HEDGE only requires a vector of actions to be stored which is near optimal in terms of space complexity. However, our algorithm is only able to cooperate in self-play around 20% of the time and is thus not a definitive answer to the problem of obtaining cooperation in self-play.

Preliminaries

Game theory. We consider a two-player strategic form game $G = (A_1, A_2, u_1, u_2)$. Each time the game is played, players 1 and 2 select an action from A_1 and A_2, respectively. The payoff of player i is given by the utility function $u_i : A_1 \times A_2 \to \mathbb{R}$. It is oftentimes convenient to encode u_i into a payoff matrix M_i where the (j, k)-th entry is $u_i(a_j, a_k)$. A *strategy* prescribes the action that a player takes when the game is played. Formally, a (mixed) strategy σ for player i is a probability distribution over A_i so that player i chooses action a with probability $\sigma(a)$. The payoff player 1 receives for following σ against an opponent action of a' is $u_1(\sigma, a') = \sum_{a \in A_1} \sigma(a) u(a, a')$.

The goal of the learner is to maximize the payoff she receives by playing G repeatedly. Of course, the payoff that the learner receives depends on her opponent, and this usually means that the learner must adapt to her opponent's strategy. In certain games, however, the learner can play a strategy that obtains the highest payoff irrespective of opponent behavior. An action $a \in A_i$ is *dominant* if $u_i(a, b) > u_i(a', b)$ for all $a' \in A_i/\{a\}$ and for all opponent actions b. A *dominant strategy* assigns a probability of 1 to playing the dominant action. In the prisoner's dilemma, defection is a dominant strategy for both players (see figure 1).

Unfortunately, it is somewhat rare for a game to have a dominant strategy. On the contrary, every finite game is guaranteed to have a *Nash equilibrium* [17]. A strategy profile (σ_1, σ_2) is a Nash equilibrium if strategy σ_1 for player 1 is a best response to strategy σ_2 for player 2 and vice versa. Specifically, this means that $u_i(\sigma_i, \sigma_{-i}) \geq u_i(\sigma', \sigma_{-i})$ for any mixed strategy σ'.[1] Note that a dominant strategy equilibrium is necessarily a Nash equilibrium, but the converse is not true.

When discussing no-regret learning algorithms, it will be useful to refer to a solution concept weaker than the Nash equilibrium. A distribution $\mathcal{D}$ over $A_1 \times A_2$ is said to be a *coarse correlated equilibrium* if for every i and $a'_i \in A_i$,

$$\mathbb{E}_{(a_1,a_2)\sim\mathcal{D}} \left(u_i(a_i, a_{-i})\right) \geq \mathbb{E}_{(a_1,a_2)\sim\mathcal{D}} \left(u_i(a'_i, a_{-i})\right) \tag{1}$$

In a coarse correlated equilibrium, players' actions are to be drawn from $\mathcal{D}$, and no player has an incentive to "ignore" the prescribed action distribution and deviate to another strategy. Nash equilibria are contained in the set of coarse correlated equilibria [13].

No-regret and Hedge. After the learner chooses an action and the game is played, she can observe the payoffs that could have been obtained by selecting a different action. If the learner chooses action a and the opponent chooses action b, the *regret* that the learner has for not having played action a' is $u(a', a) - u(a, b)$. After T iterations, the total regret of the learner from playing the sequence of actions $a^1, \ldots, a^T$ is defined to be

$$\max_{a \in A_i} \sum_{j=1}^{T} u_i(a, a^j_{-i}) - u_i(a^j, a^j_{-i}) \tag{2}$$

The definition of regret allows the learner to compare her payoff to what she could have received by playing some fixed action, under the assumption that the opponent would have played the same actions on each iteration. A learning algorithm is said to be *no-regret* if for any sequence of opponent actions

$$\lim_{T\to\infty} \frac{1}{T} \left(\max_{a \in A_i} \sum_{j=1}^{T} u_i(a, a^j_{-i}) - u_i(a^j, a^j_{-i}) \right) \leq 0 \tag{3}$$

In other words, the time average of regret is nonpositive in the long run.

Perhaps the most well known algorithm for achieving no-regret is Hedge , which was introduced in [10]. Hedge is closely related to the Weighted Majority algorithm [15], a meta-algorithm that has been successfully applied in a variety of fields [1] and is the basis of the AdaBoost algorithm [9]. In the setup of the Weighted Majority algorithm, it is assumed that the learner has access to n "experts" each recommending an action to the learner. The learner

[1] A subscript of $-i$ refers to the set of players other than i.

follows the advice of the chosen expert and is able to observe (in hindsight) the outcome had she listened to a different expert. The Weighted Majority algorithm maintains a probability distribution over experts, placing more weight on experts who consistently produce favorable results. In the setting of learning a game, the experts recommend an action and the learner maintains a mixed strategy over the possible actions. HEDGE gives a precise way in which the mixed strategy is updated after each iteration. To begin, let $p^{(0)}$ be the uniform distribution over the n possible actions. After the t-th iteration of the game, the strategy $p^{(t+1)}$ is formed by setting

$$p_j^{(t+1)} = \frac{p_j^{(t)} \beta^{u_i(a_j, a_{-i})}}{\sum_{k=1}^{N} p_i^{(t)} \beta^{u_i(a_k, a_{-i})}} \tag{4}$$

where $\beta > 1$ is a tuned parameter. The main result of [10] is showing that HEDGE is a no-regret learning algorithm for playing a game. In particular, if β is set properly, then [10] shows that the average regret per iteration is $O\left(\sqrt{\frac{\ln n}{T}}\right)$ after T iterations. The logarithmic dependence on the number of actions n means that HEDGE will enjoy fast convergence to a state of no-regret even when the game is large. Notice that HEDGE only requires the learner to update her probability distribution each round, and so HEDGE has an $O(n)$ space complexity. Since any algorithm storing the full payoff matrix requires $\Omega(n^2)$ space (at least when the game is symmetric), HEDGE obtains no-regret performance even without complete knowledge of the game being played.

Computing equilibria. While equilibria are useful tools for making theoretical predictions about how agents will play a game, the practical relevance of these predictions certainly depends on the complexity of computing such an equilibrium. It is suspected that there is no efficient algorithm for exactly computing a Nash equilibrium in a general game [6]. Many decision problems related to Nash equilibria are known to be NP-hard (e.g., "Are there at least two Nash equilibria in a given game?") [12]. The current approach to compute a Nash equilibrium is to apply the Lemke-Howson algorithm [14], which requires an exponential run-time for certain games. In the special case of a zero-sum game,[2] a Nash equilibrium can be computed by solving a linear program. Still, neither approach seems to capture the process that a realistic agent might use to learn and play a game.

Given the challenges of exactly computing a Nash equilibrium, one might hope that a no-regret learning algorithm like HEDGE approximates a Nash equilibrium in the limit. Unfortunately, [5] provides a 3×3 game where HEDGE fails to converge to any Nash equilibrium. Note that are special cases where the algorithm will converge to a Nash equilibrium, such as zero-sum games [10]. Not all is lost, since the following result shows that HEDGE will in general converge to a coarse correlated equilibrium:

Theorem 1. *Let G be any two-player game. If both players choose strategies according to* HEDGE*, then the time-average of the strategy profiles will converge to a coarse correlated equilibrium of G.*

Proof. The proof is adapted from [13]. Let $\mathcal{D}_t$ be the joint probability distribution over both players' actions on iteration t and let $\mathcal{D}$ be the time average of the $\mathcal{D}_t$. Then

$$\mathbb{E}_{a\sim\mathcal{D}}(u_i(a)) = \frac{1}{T}\sum_{i=1}^{T} \mathbb{E}_{a\sim\mathcal{D}_t}(u_i(a)) \tag{5}$$

Fix $a_i{}' \in A_i$. Since HEDGE is no-regret, T can be chosen large enough so that for any $\varepsilon > 0$, the regret of not choosing a_i' is less than ε. Thus,

$$\frac{1}{T}\sum_{i=1}^{T} \mathbb{E}_{a\sim\mathcal{D}_t}(u_i(a)) \geq \left(\frac{1}{T}\sum_{i=1}^{T} \mathbb{E}_{a\sim\mathcal{D}_t}(u_i(a_i', a_{-i}))\right) - \varepsilon \tag{6}$$

$$= \mathbb{E}_{a\sim\mathcal{D}}(u_i(a_i', a_{-i})) - \varepsilon \tag{7}$$

Since ε can be chosen arbitrarily small, we have verified the condition (1) for $\mathcal{D}$ to be a coarse correlated equilibrium. ■

Note that time-averaged convergence does not imply that the actual strategy profiles converge. In fact,

[2] A zero-sum game has the property $u_i(a_i, a_{-i}) = -u_i(a_i, a_{-i})$.

		Player 2	
		H	**T**
Player 1	**H**	$1, -1$	$-1, 1$
	T	$-1, 1$	$1, -1$

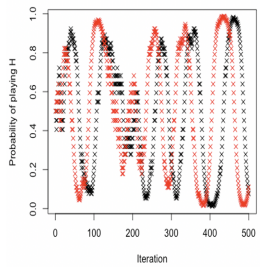


Figure 2: Play-by-play behavior can be chaotic in the matching pennies game when both players use HEDGE. *In black, the probability that player 1 plays H; red, the probability that player 2 plays H. $\beta = 1.1$.*

there are games for which the play-by-play behavior of HEDGE can be chaotic [18]. Recently, [2] showed that when two copies of HEDGE play against each other in games without pure strategy Nash equilibria, the strategy profiles are actually repelled from the Nash equilibrium. Figure 2 shows an example of this in the matching pennies game (where the only Nash equilibrium is to randomize between H and T). Instead of randomizing between H and T, players appear to take turns between periods of loses and gains. [2] notes that this non-covergent behavior can be corrected by choosing the update parameter β_t to be decreasing in t such that $\sum_{t=1}^{\infty}(\beta_t - 1) < \infty$. Although our experiments confirm this, we note that this may result in the strategy profiles converging to something other than the Nash equilibrium.

Methods

A major drawback of HEDGE is that it assumes the opponent is adversarial and completely unresponsive. As we saw earlier, the no-regret framework is inadequate for realistic situations where the learner's decisions impact the opponent's strategy. Even when two copies of HEDGE play against each other, both learners are responsive and update their strategies based on the actions of their opponent. Our goal is thus to alter HEDGE in the hope of obtaining an algorithm that can best respond to simple reactive opponents.

If the opponent is reactive, then she might change her strategy at time $t+1$ based on the action of the learner at time t. Thus, a favorable payoff for the learner at time $t+1$ may in part be the result of choosing a good action at time t. To incorporate this idea into HEDGE, we update the action chosen at time t with the payoff at time $t+1$. Our proposed algorithm, which we will call Delayed Update Hedge (DU-HEDGE), uses a new parameter α to control how much the action $a^{(t)}$ impacts the weight update at time $t+1$. The learner can then tune both parameters α and β depending on how responsive she thinks her opponent is. The pseudocode for DU-HEDGE is given in algorithm 1. Note that on the first iteration, the weights are updated according to the usual HEDGE rule (since $a^{(0)}$ is undefined).

Algorithm 1: DU-HEDGE(α, β, A)

Initialize $w^{(1)}(a) = 1/|A|$ for all $a \in A$
Sample action $a^{(1)}$ from $w^{(1)}$
Set $w^{(2)}$ using the HEDGE update rule
for $t = 2, \ldots, T$ **do**
  Normalize $w^{(t)}$ to obtain distribution $p^{(t)}$
  Sample action $a^{(t)}$ from $p^{(t)}$
  $w^{(t+1)}(a^{(t-1)}) = w^{(t)}(a^{(t-1)})\beta^{u_1(a^{(t)},\ b^{(t)})}$
  for $a' \neq a^{(t-1)}$ **do**
    $w^{(t+1)}(a') = w^{(t)}(a')\alpha^{u_1(a',\ b^{(t)})}$
  end
end

Algorithmically, DU-HEDGE maintains many of the favorable properties of HEDGE. For instance, the memory complexity is linear in the number of actions, since only $w^{(t)}$ is required to form $w^{(t+1)}$. In addition, updating the weight vector only requires multiplication which makes it computationally simple. Contrary to HEDGE, we do not expect DU-HEDGE to be no-regret for all α and β, since otherwise the algorithm would be unable to cooperate in games such as the prisoner's dilemma. Still, in the case $\alpha = \beta$ the following result shows that DU-HEDGE will not get exploited by an unresponsive opponent.

Theorem 2. *Let G be a game for which the learner has n available actions, one of which is dominant.*

Suppose that the opponent is unresponsive and always plays b. Furthermore, suppose that G is such that

$$\frac{1}{2}\left(u_1(a,b) + \min_{a' \in A} u_1(a',b)\right) \geq \max_{a' \in A/\{a\}} u_1(a',b) \tag{8}$$

where a is the dominant action. Then if the learner plays this game using DU-HEDGE *with $\alpha = \beta$ the mixed strategies will converge in probability to a.*

Note that in the case $n = 2$, condition (8) is trivially satisfied.

Proof. We prove the general case of $n > 2$ and note that the case of $n = 2$ is shown similarly. Let $a_{\min} = \text{argmin}_{a \in A} u(a,b)$. Assume for the moment that there is a t such that $w^{(t)}(a) \geq w^{(t)}(a')$ for all $a' \neq a$. Then there are three distinct possibilities:

1. $a^{(t-1)} = a$ and $a^{(t)} = a$. Then for any a',

$$w^{(t+1)}(a) = w^{(t)}(a)\beta^{u_1(a,b)} \tag{9}$$
$$> w^{(t)}(a)\alpha^{u_1(a',b)} \tag{10}$$
$$= w^{(t+1)}(a') \tag{11}$$

2. $a^{(t-1)} = a$ but $a^{(t)} \neq a$. If $a^{(t)} = a'$, then

$$w^{(t+1)}(a) = w^{(t)}(a)\beta^{u_1(a',b)} \tag{12}$$
$$\geq w^{(t)}(a')\alpha^{u_1(a',b)} \tag{13}$$
$$= w^{(t+1)}(a') \tag{14}$$

   If $a^{(t)} \neq a'$, then

$$w^{(t+2)}(a) \geq w^{(t)}(a)\beta^{u_1(a_{\min},b)}\alpha^{u_1(a,b)} \tag{15}$$
$$= w^{(t)}(a)\alpha^{u_1(a_{\min},b)+u_1(a,b)} \tag{16}$$
$$\geq w^{(t)}(a)\alpha^{2u_1(a',b)} \tag{17}$$
$$= w^{(t+2)}(a') \tag{18}$$

3. If $a^{(t-1)} \neq a$, then clearly $w^{(t+1)}(a) \geq w^{(t+1)}(a')$ for any a'.

For any a', define the stochastic process $X_t = w^{(t)}(a)/w^{(t)}(a')$. Since $X_1 = 1$, the above cases show that if $X_{t+1} < X_t$ then $X_{t+2} \geq X_t$ and that $X_{t+1} > X_t$ with a non-zero probability (i.e., a is selected twice in a row). Thus $X_t \to \infty$ with probability 1 which means that the probability of selecting a also approaches 1. ■

Since most games do not have a dominant strategy, theorem 2 only applies to a narrow subset of games. However, in experiments we will show that DU-HEDGE with $\alpha = \beta$ is similar to HEDGE even in games without dominant strategies. The interesting case is thus $\alpha > \beta$, so that the learner believes her previous actions greatly influence the payoff in future rounds.

Results

Prisoner's Dilemma. We test DU-HEDGE on the prisoner's dilemma game as shown in figure 1. Figure 3 (A) gives a table showing the average payoff of DU-HEDGE when playing against various opponent strategies. When $\alpha = \beta$, the algorithm performs similarly to HEDGE, which will achieve an average payoff of 1 against any strategy (reactive or not) in the prisoner's dilemma. Figure 3 (C) plots the performance by iteration of HEDGE and DU-HEDGE against an opponent strategy of always defecting and shows that HEDGE converges faster to the best response. However, when $\beta < \alpha$, we see that DU-HEDGE is consistently able to cooperate when playing against the tit-for-tat strategy. Figure 3 (B) plots the performance of DU-HEDGE against tit-for-tat for various values of β, showing that $\alpha = 1.1$, $\beta \approx 1.06$ is sufficient to induce cooperative behavior. As expected, DU-HEDGE is unable to cooperate with more complicated opponent strategies. In the 2-stage grim trigger (GT), the opponent defects for 2 rounds each time the learner defects, and thus only cooperates after the learner takes two consecutive rounds of punishment. Since DU-HEDGE only updates weights based on the previous action, it is unable to detect strategies that depend on actions further in the past.

When $\alpha = 1.1$ and $\beta = 1.01$, DU-HEDGE shows signs of cooperative behavior even in self-play. However, the performance is somewhat poorer against unresponsive opponents for these parameter values. We would like to induce cooperation in self-play for the parameter values $\alpha = 1.1, \beta = 1.05$ which appear to yield more stable results against unresponsive opponents. To do this, we introduce a heuristic that forces the players to explore more of the game space.

First, divide the game into chunks of N rounds, which we will call periods. Every period, the player compares her total payoff for that period, U_{pres}, to her total payoff for the *previous* period, U_{prev}. If $U_{pres} < cU_{prev}$, then the player "resets" her strategy by selecting a new probability distribution uniformly at random. The idea is thus to force the learner to choose a new strategy when the payoff from her current strategy begins to decline. For experiments, we choose $N = 25, c = 0.9, \alpha = 1.1, \beta = 1.05$ and run 1000 trials each having 2000 rounds. For tit-for-tat, all C, and all D, adding the heuristic did not change the average payoff. In self-play, however, the heuristic led to cooperative behavior in 208 of the 1000 runs. For comparison, cooperation occurred in only 17 of the 1000 trials without the heuristic. See figure 3 (D) for an example of what the dynamics look like when self-play leads to cooperation. Note the heuristic also allows the learner to be flexible in the event that the opponent occasionally switches strategies. For example, DU-Hedge may get exploited by an adversary who begins by playing tit-for-tat, but then switches to defection after hundreds of rounds. The heuristic with N set reasonably low will allow the learner to quickly detect this change and react accordingly.

Shapley game. As mentioned earlier, [5] presented a game for which Hedge will not converge (in the time-averaged sense) to a Nash equilibrium. The 3×3 game, which is shown in figure 4, is closely related to the game presented by Shapley in 1964 to prove that fictitious play also does not converge to a Nash equilibrium [20]. Note that the only Nash equilibrium in the Shapley game is to mix between L, C and R with equal probabilities. We explore the behavior of Hedge and DU-Hedge in self-play

A

Opponent Strategy	$\alpha = \beta = 1.1$	$\alpha = 1.1, \beta = 1.05$	$\alpha = 1.1, \beta = 1.01$	Optimal
All C	4.976	4.909	**3.827**	5.000
All D	0.978	0.963	**0.739**	1.000
Tit-for-tat	1.041	**2.943**	**2.987**	3.000
Self-play	1.039	1.096	**1.930**	3.000
2-Stage GT	1.011	0.985	0.984	3.000

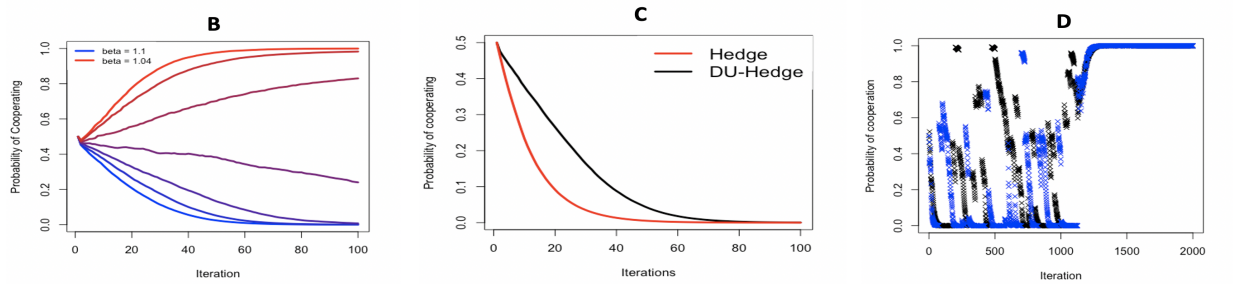
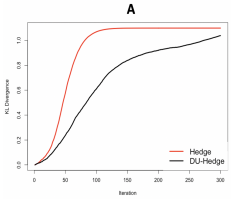


Figure 3: Experiments with DU-Hedge *on the prisoner's dilemma game.* ***(A)*** *The average payoff for* DU-Hedge *against* 5 *different opponent strategies. Mean of* 100 *trials each of which played the game* 100 *times. Numbers in red indicate that the performance of* DU-Hedge *is significantly worse than* DU-Hedge*, while numbers in green indicate performance significantly better than* Hedge*.* ***(B)*** *Probability of cooperating (average) by iteration for values of* β *ranging from* 1.1 *to* 1.04 *(decreasing by* 0.1*).* ***(C)*** *Proability of cooperating (average) against an always defecting opponent, comparison of the two algorithms for* $\alpha = \beta = 1.1$*.* ***(D)*** *An example where the search heuristic lead to cooperation in self-play.*

Shapley's Game

		Player 2		
		L	**R**	**C**
Player 1	**L**	$0,0$	$1,2$	$2,1$
	R	$2,1$	$0,0$	$1,2$
	C	$1,2$	$2,1$	$0,0$

A



B

Algorithm	Average payoff
HEDGE $\beta = 1.1$	1.42
DU-HEDGE $\alpha = \beta = 1.1$	1.38
DU-HEDGE $\alpha = 1.1$, $\beta = 1.05$	1.24

*Figure 4: The Shapley game as given in [5]. **(A)** KL divergence from the unique Nash equilibrium (mean of 100 trials). **(C)** Average payoff comparisons (mean of 100 trials with 300 rounds of the game).*

of the Shapley game. For each iteration, we compute the Kullback–Leibler (KL) divergence between the strategy $p^{(t)}$ of the learning algorithm and the Nash equilibrium $(1/3, 1/3, 1/3)$. Figure 4 (B) confirms that both HEDGE and DU-HEDGE fail to converge (in self-play) to the Nash equilibrium of the Shapley game. Interestingly, the rate of divergence appears to be slower (and much less structured) for DU-HEDGE. Although failing to converge to Nash equilibria may be theoretically unsatisfying, the payoffs achieved by the learning algorithms can be better than the average payoff in a Nash equilibrium. Note that the average payoff in the Nash equilibrium of the Shapley game is 1. For comparison, the average payoffs achieved by HEDGE and DU-HEDGE are shown in figure 4. Note that the payoffs of HEDGE and DU-HEDGE with $\alpha = \beta$ are quite close, justifying our belief that these algorithms are essentially the same for a wide class of games.

Discussion

Our experiments show that DU-HEDGE with $\alpha > \beta$ is able to cooperate against simple opponent strategies. With the addition of the search heuristic, DU-HEDGE is able to cooperate with itself around 20% of the time (in trials with 2000 iterations). To the best of our knowledge, this is the only learning algorithm that can attain cooperative behavior with the space and time complexity of HEDGE. Still, it required a carefully crafted heuristic to get self-play cooperation in a minority of trials, which means that there is significant room for improvement. The development of improved algorithms that are able to cooperate in self-play is thus an important area for future research.

There are a number of improvements that can be made to DU-HEDGE. It would be interesting to see if the parameters α and β can themselves be learned based on feedback from the opponent. This would allow the learner to use updates similar to HEDGE when the opponent is unresponsive and to be mindful of past actions otherwise. Additionally, DU-HEDGE can be adjusted to rely on more than just one action in the past. This would potentially allow the algorithm to cooperate against the 2 stage grim trigger. However, this may over-complicate the algorithm to the point where theoretical results are impossible. Finally, we suspect that there are results stronger than theorem 2 regarding the behavior of DU-HEDGE. When $\alpha = \beta$, we saw that the behavior of DU-HEDGE was nearly identical to that of HEDGE, suggesting that there might be a way to adapt the results from [10].

Our paper also brings forth some other interesting unanswered questions about the behavior of HEDGE. The recent work of [2, 18] has begun to investigate the chaotic play-by-play behavior of HEDGE. The observed behavior appears to vary widely between different games and are not well understood. Future research could be aimed at a more general classification of the chaos that can appear. Another question is classifying the games for which HEDGE fails to converge to a Nash equilibrium. To the best of our knowledge, the Shapley game presented in [5] is the only known counterexample. It would thus be interesting to see if more counterexamples can be found to determine if the Shapley game is the exception or the norm. On a related note, the behavior of HEDGE for very large games appeared to be unstudied. Since HEDGE achieves a logarithmic regret bound with linear space, it is likely to be a very useful tool in learning large games.

References

[1] S. Arora, E. Hazan, and S. Kale. The multiplicative weights update method: A meta-algorithm and applications. *Theory of Computing*, pages 121–164, 2012.

[2] J. Bailey and G. Piliouras. Mulitplicative weights update in zero-sum games. In *ACM EC*, 2018.

[3] D. Banjeree and S. Sen. Reaching pareto-optimality in prisoner's dilemma using conditional joint action learning. *Autonomous Agent Multi-Agent Systems*, pages 91–108, 2007.

[4] Y. Chang. No regrets about no-regret. *Artificial Intelligence*, pages 434–439, 2007.

[5] C. Daskalakis, R. Frongillo, C. Papadimitriou, G. Pierrakos, and G. Valiant. On learning algorithms for nash equilibria. In *Symposium on Algorithm Game Theory (SAGT)*, 2010.

[6] C. Daskalakis, P. Goldberg, and C. Papadimitriou. The complexity of computing a nash equilibrium. In *Proceedings of the Thirty-Eighth Annual ACM Symposium on Theory of Computing*, STOC '06, page 71–78, New York, NY, USA, 2006. Association for Computing Machinery.

[7] D. de Farias and N. Megiddo. How to combine expert (or novice) advice when actions impact the environment. *Advances in Neural Information Processing Systems, MIT Press*, 2004.

[8] M. Faure, P. Gaillard, B. Gaujal, and V. Perchet. Online learning and game theory: a quick overview with recent results and applications. *ESIAM: Proceedings and Surveys*, pages 246–271, 2015.

[9] Y. Freund and R. Schapire. A decision-theoretic generalization of on-line learning and an application to boosting. *Journal of Computer and System Sciences*, pages 119–139, 1997.

[10] Y. Freund and R. Schapire. Adaptive game playing using multiplicative weights. *Games and Economic Behavior*, 29:79–103, 1999.

[11] D. Fudenberg and D. Levine. Consistency and cautious fictitious play. *Journal of Economic Dynamics and Control*, pages 1065–1089, 1995.

[12] I. Gilboa and E. Zemel. Nash and correlated equilibria: Some complexity considerations. *Games and Economic Behavior*, pages 80–93, 1989.

[13] H. Heidari. No-regret learning in games. unpublished.

[14] C. Lemke and J. Howson. Equilibrium points of bimatrix games. *SIAM Journal on Applied Mathematics*, pages 413–423, 1964.

[15] N. Littlestone and M. Warmuth. The weighted majority algorithm. *Information and Computation*, pages 212–261, 1994.

[16] R. Marks. *Repeated games and finite automata*. Australian Graduate School of Management, University of New South Wales, 1990.

[17] J. Nash. Equilibrium points in n-person games. *Proceedings of the National Academy of Sciences*, 36(1):48–49, 1950.

[18] G. Palaiopanos, I. Panageas, and G. Piliouras. Multiplicative weights update with constant step-size in congestion games: Convergence, limit cycles and chaos. In *31st Conference on Neural Information Processing Systems (NIPS)*, 2017.

[19] K. Schlag and A. Zapechelnyuk. On the impossibility of achieving no regrets in repeated games. *Journal of Economic Behavior and Organization*, pages 153–158, 2012.

[20] L. Shapley. Some topics in two-person game. *Advances in Game Theory*, pages 1–29, 1964.